MultiTenantPlatform

Software Development Life Cycle Document

Document ID	MTP-SDLC-001
Version	1.0
Status	Released
Classification	Confidential — Internal Use Only
Project	MultiTenantPlatform (Multi-Tenant Modular SaaS)
Technology Stack	FastAPI · PostgreSQL · React (Vite) · Docker · Alembic
Date	March 2026
Owner	Engineering Lead

This document describes the complete software development lifecycle, architecture, processes, and governance for the MultiTenantPlatform project. It serves as the primary reference for engineering, QA, DevOps, and product teams.

1. Introduction

1.1 Purpose

This Software Development Life Cycle (SDLC) document describes the end-to-end methodology, processes, tooling, and governance adopted by the MultiTenantPlatform engineering team. It is intended to ensure consistent, high-quality delivery across all phases of the product lifecycle — from requirements gathering through deployment, monitoring, and maintenance.

1.2 Scope

This document covers the MultiTenantPlatform system: a production-grade, multi-tenant SaaS backend built with FastAPI and a React frontend. The system provides tenant management, role-based access control (RBAC), modular domain plugins (school, hospital, HRMS, e-commerce), subscription management, feature toggles, and an audit trail.

In scope:

- Backend API (FastAPI, PostgreSQL, Alembic)
- Frontend web application (React, Vite, TypeScript)
- Infrastructure (Docker, docker-compose, Render deployment)
- CI/CD pipeline, testing strategy, and operational processes

Out of scope: mobile applications, third-party payment gateway integrations, and data warehousing.

1.3 Definitions and Acronyms

Term / Acronym	Definition
SDLC	Software Development Life Cycle — the structured process for planning, creating, testing, and delivering software
RBAC	Role-Based Access Control — restricting system access based on role assignments
JWT	JSON Web Token — a compact, self-contained bearer token used for authentication and context switching
Tenant	An isolated organisational instance within the platform (e.g. a school, hospital)
DFD	Data Flow Diagram — depicts data movement between processes and stores
ORM	Object-Relational Mapper — SQLAlchemy in this project
UAT	User Acceptance Testing — validation by business stakeholders before release
SLA	Service Level Agreement — contractual performance and availability commitments

1.4 References

- classdiagram.html — UML Class Diagram (Domain Model)
- usecase.html — UML Use-Case Diagram
- dataflow.html — Level 1 Data Flow Diagram (Yourdon-DeMarco)
- README.md — Developer setup and deployment guide
- render.yaml — Production deployment specification

2. Project Overview

2.1 Product Vision

MultiTenantPlatform is a cloud-native, modular SaaS backend designed for organisations that need to manage multiple isolated tenants — each with their own members, roles, permissions, subscriptions, and domain modules — under a single platform umbrella. The architecture prioritises security, extensibility, and observability.

2.2 System Architecture Summary

The platform follows a layered architecture:

- Presentation Layer: React (Vite + TypeScript) SPA communicating via REST API
- API Layer: FastAPI application exposing versioned endpoints under /api/v1
- Middleware Layer: Rate limiting, CORS, request-ID injection, JWT validation
- Business Layer: Domain-specific endpoint routers (auth, tenants, roles, modules, audit)
- Data Layer: PostgreSQL 16 via SQLAlchemy ORM; Alembic for schema migrations
- Infrastructure: Docker containers orchestrated via docker-compose; production on Render

2.3 Key Capabilities

Capability	Description
Multi-tenancy	Full tenant isolation with slug-based identification, industry typing, and localization (country, currency, timezone, language)
RBAC	Hierarchical roles with atomic permissions; Admin role treated as tenant superuser; per-endpoint permission enforcement
JWT Authentication	HS256 tokens with token_version for instant revocation; tenant context switching via /auth/switch-context
Module System	Pluggable domain modules (school, hospital, HRMS, e-commerce) enabled per tenant
Subscription Management	Basic/Pro/Enterprise plans with expiry tracking; old plans deactivated on upgrade
Feature Toggles	Per-tenant key/bool feature flags for granular rollout without code deployment
Audit Trail	Append-only audit log capturing actor, resource, action, IP, request-ID, and user-agent
Rate Limiting	Sliding-window in-memory rate limiter; configurable per endpoint (e.g. 5/5 min on register)

3. SDLC Methodology

3.1 Selected Model

MultiTenantPlatform adopts an Agile Iterative development model with two-week sprints. The model combines the predictability of phase-gated planning with the flexibility of iterative delivery, enabling continuous stakeholder feedback while maintaining engineering rigour.

3.2 Phase Overview

Phase	Objective	Key Outputs
1. Requirements	Elicit, analyse, and document functional and non-functional requirements	User stories, acceptance criteria, API contract, domain model
2. System Design	Define architecture, data model, API surface, and security controls	UML diagrams (class, use-case, DFD), DB schema, OpenAPI spec
3. Implementation	Sprint-based feature development with daily standups and PR reviews	Source code, Alembic migrations, Docker images
4. Testing	Unit, integration, smoke, and end-to-end testing across all layers	Test reports, coverage metrics, bug trackers
5. Deployment	Containerised CI/CD pipeline delivering to staging then production	Docker images, render.yaml config, migration scripts
6. Maintenance	Monitor, patch, and iteratively improve the running system	Incident reports, patch releases, dependency updates

4. Requirements

4.1 Stakeholders

Stakeholder	Role	Interest
Platform Admin	Creates and manages tenants	Reliable tenant provisioning and user management
Tenant Admin	Manages members and modules within a tenant	RBAC configuration, subscription control, audit visibility
Tenant Member	Uses domain modules assigned to their role	Reliable access to permitted resources
Engineering Team	Builds and maintains the platform	Clean architecture, testability, clear processes
DevOps / Infra	Deploys and monitors the system	Stable CI/CD, observable infrastructure, easy rollback

4.2 Functional Requirements

4.2.1 Authentication & Session

- FR-001: Users must be able to register with email and password (min. 8 characters, bcrypt hashed)
- FR-002: Login must issue a HS256 JWT with sub, ver, and optional tenant_id claims
- FR-003: Token version must be incremented on logout/password-change to invalidate all sessions
- FR-004: The /auth/switch-context endpoint must issue a tenant-scoped JWT
- FR-005: Password change must re-validate the current password before accepting the new one

4.2.2 Tenant Management

- FR-010: A new tenant must auto-create an Admin role and assign the creator as Admin member
- FR-011: Tenant slugs must be globally unique and contain only lowercase letters, numbers, and dashes
- FR-012: Members can be added by email or user_id with a specified role
- FR-013: Re-adding a deactivated member must reactivate them without creating a duplicate row

4.2.3 RBAC

- FR-020: The Admin role must bypass all permission checks (tenant superuser)
- FR-021: Non-admin roles must hold an explicit RolePermission row to access guarded endpoints
- FR-022: Default permissions must be seeded from DEFAULT_PERMISSIONS on tenant creation

4.2.4 Modules & Subscriptions

- FR-030: Modules (school, hospital, HRMS, e-commerce) must be enabled per tenant independently
- FR-031: Re-enabling an existing TenantModule must update is_enabled, not create a duplicate
- FR-032: Creating a new subscription must deactivate all previous subscriptions for the tenant
- FR-033: Feature toggles must support arbitrary string keys per tenant

4.2.5 Audit Logging

- FR-040: All mutating operations must write an audit log entry before returning a response
- FR-041: Audit logs must store user_id, tenant_id, action, resource, IP, request_id, user-agent, and JSON details
- FR-042: Audit log reads must be scoped to the authenticated tenant (cross-tenant reads must return 403)

4.3 Non-Functional Requirements

ID	Category	Requirement	Target
NFR-001	Performance	API response time for tenant-scoped read operations	< 200ms p95
NFR-002	Availability	Platform uptime (excluding planned maintenance)	99.5% monthly
NFR-003	Security	Passwords stored with bcrypt (min. cost 12); no plaintext storage	Mandatory
NFR-004	Security	JWT max validity; bcrypt limited to 72 bytes	10 days default
NFR-005	Scalability	Support >= 100 concurrent tenants without schema changes	Architecture
NFR-006	Maintainability	All schema changes managed via Alembic versioned migrations	Mandatory
NFR-007	Observability	Every response must include x-request-id header for tracing	Mandatory
NFR-008	Compliance	Rate limiting on auth endpoints to prevent brute-force attacks	5 req/5 min

5. System Design

5.1 Architecture Decisions

Decision	Choice	Rationale
Web framework	FastAPI (Python)	Async-capable, auto-generates OpenAPI docs, strong Pydantic integration for validation
Database	PostgreSQL 16	ACID compliance, mature ecosystem, native Alembic support, JSON column availability
ORM	SQLAlchemy (sync)	Mature, expressive, integrates with Alembic migrations; sync mode simplifies dependency injection
Auth token	JWT HS256	Stateless; tenant_id embeddable; token_version enables revocation without a token blacklist
Frontend framework	React + Vite	Fast HMR, tree-shaking, TypeScript support; shadcn/ui + Tailwind for component system
Containerisation	Docker	Reproducible builds; docker-compose for local dev with Postgres + API; Render for production
Schema migrations	Alembic	Version-controlled, reversible migrations run automatically on container startup

5.2 Data Model Summary

The domain model consists of 11 core entities. See classdiagram.html for the full UML class diagram.

Entity	Group	Primary Responsibility
User	Identity	Global identity; credentials; token_version for session revocation
Tenant	Tenancy	Isolated organisation with slug, industry, and localization attributes
UserTenant	Tenancy	Membership join; carries role assignment; enables context switching
Role	RBAC	Named permission bundle scoped to a tenant; Admin = superuser
Permission	RBAC	Atomic capability string (e.g. attendance:write); global registry
RolePermission	RBAC	Many-to-many join checked at runtime by require_permission()
Module	Modules	Domain plugin descriptor; globally registered
TenantModule	Modules	Activates a module for a specific tenant; supports re-enabling

Subscription	Billing	Plan tier with expiry; old plans deactivated on upgrade
FeatureToggle	Config	Per-tenant key/bool flags for zero-deployment feature rollout
AuditLog	Observability	Append-only record of actions with IP, request_id, and JSON payload

5.3 API Design

All endpoints are versioned under /api/v1. Authentication uses Bearer tokens. Tenant-scoped endpoints require a context-switched token containing tenant_id.

Router	Prefix	Key Endpoints
Auth	/auth	POST /register, /login, /switch-context, /logout, /change-password · GET /me
Tenants	/tenants	POST /, GET /, GET /{id}, POST /{id}/members, GET /{id}/members
Roles	/roles, /permissions	POST /roles, GET /roles, POST /roles/{id}/permissions, GET /permissions
Modules	/modules, /tenants/{id}/*	POST/GET /modules; /tenants/{id}/modules, /subscriptions, /features
Audit	/audit-logs	GET /audit-logs (tenant_id, user_id, limit query params)

5.4 Security Design

- Authentication: bcrypt password hashing (passlib); JWT HS256 signed with configurable SECRET_KEY
- Session revocation: token_version integer on User incremented at logout/password-change; JWT ver claim checked on every request
- Tenant isolation: ensure_path_tenant() enforces that path tenant_id matches RequestContext.tenant_id; cross-tenant reads return 403
- RBAC: require_permission() dependency injected at router level; Admin role bypasses checks; all others require RolePermission row
- Rate limiting: SlidingWindowRateLimiter (in-memory, thread-safe) on /register (5/5 min), /login (10/5 min), /change-password (10/10 min)
- Input validation: Pydantic v2 schemas with EmailStr, Field(min_length=8), enum constraints on all request bodies
- bcrypt safety: validate_bcrypt_password_length() rejects passwords > 72 bytes to prevent bcrypt truncation vulnerability

6. Implementation

6.1 Development Environment

Tool	Version / Notes
Python	3.11 (as per Dockerfile base image python:3.11-slim)
Node.js	>=18 (required by Vite and frontend toolchain)
Docker	Docker Desktop / Docker Engine; docker-compose v2
PostgreSQL	16 (Alpine image; mapped to host port 5433 in dev)
IDE	Any; VS Code with Pylance + ESLint recommended
Package manager (BE)	pip (requirements.txt)
Package manager (FE)	npm (package-lock.json committed)

6.2 Local Setup

Full stack (recommended) — runs Postgres + API in Docker, frontend via Vite:

- Clone the repository and run ./scripts/dev.sh
- API available at http://127.0.0.1:8080; Frontend at http://127.0.0.1:8888

Backend only (no Docker):

- Create .env from .env.example; set DATABASE_URL
- pip install -r requirements.txt
- python -m alembic -c alembic/alembic.ini upgrade head
- uvicorn app.main:app --reload --port 8000

6.3 Project Structure

Path	Purpose
app/api/v1/endpoints/	FastAPI route handlers for auth, tenants, roles, modules, audit
app/core/	Config, security (JWT/bcrypt), dependencies (auth guards, RBAC), rate limiter
app/models/models.py	All SQLAlchemy ORM models (11 entities)
app/schemas/schemas.py	Pydantic v2 request/response schemas
app/services/	Service layer: auth_service (register/login/switch), audit_service (write_audit_log)
app/db/session.py	SQLAlchemy engine, session factory, declarative Base, get_db() dependency

alembic/	Database migration scripts; env.py reads DATABASE_URL from settings
frontend/src/features/	React feature modules: auth, tenants, roles, modules, app shell, audit
frontend/src/shared/	Shared API client (axios), auth store (zustand), types, UI primitives
scripts/	dev.sh (full-stack dev), start.sh (container entrypoint), smoke_test_http.py

6.4 Coding Standards

- Python: PEP 8; type hints on all function signatures; no print() in production code
- FastAPI: dependencies via Depends(); never mutate shared state outside a database session
- SQLAlchemy: always use db.flush() before accessing generated IDs within a transaction
- TypeScript: strict mode off for this project (tsconfig); noImplicitAny: false; prefer explicit types on API responses
- React: feature-based folder structure; lazy-loaded routes; tanstack-query for server state
- Commits: Conventional Commits format (feat:, fix:, chore:, docs:)
- PR reviews: minimum 1 approval; no direct pushes to main

7. Testing Strategy

7.1 Testing Levels

Level	Scope	Tools	Ownership
Unit	Individual functions/services in isolation; mock DB	pytest, pytest-mock	Developer
Integration	API endpoint + real DB; Alembic-migrated test DB	pytest, httpx.AsyncClient	Developer
Smoke	End-to-end HTTP flows on a live server (scripts/smoke_test_http.py)	Python urllib (stdlib)	CI / DevOps
Frontend unit	React component and hook tests	Vitest, Testing Library	Developer
E2E (browser)	Full user journey in real browser	Playwright	QA / Developer
UAT	Stakeholder validation against acceptance criteria	Manual	Product Owner

7.2 Critical Test Scenarios

7.2.1 Authentication & Security

1. Register → Login → get /auth/me → verify user object returned
2. Login with wrong password → 401 returned
3. Logout → attempt to use old token → 401 returned (token_version check)
4. Attempt to use account token on tenant-scoped endpoint → 403 (no tenant context)
5. Password > 72 bytes → 400 bad request (bcrypt safety)
6. Register 6 times from same IP within 5 min → 429 rate limited

7.2.2 RBAC & Multi-tenancy

7. Admin can add members, create roles, assign permissions
8. Non-admin with missing permission returns 403
9. Cross-tenant read attempt (wrong tenant_id in path) → 403
10. Manager (with tenant:manage) can add members — RBAC delegation

7.2.3 Smoke Test Coverage (smoke_test_http.py)

The existing smoke test validates the following full workflow:

- Register admin user → login → create tenant → switch context
- Create Manager role → assign tenant:manage permission
- Register second user → add as member with Manager role
- Manager switches context → lists members → adds a third user
- Assert HTTP 201 with correct role_name on membership response

7.3 Coverage Targets

Layer	Coverage Target	Measurement Tool
Backend (Python)	>= 80% line coverage	pytest-cov
Frontend (TypeScript)	>= 70% branch coverage	Vitest (v8 provider)
Critical paths (auth, RBAC)	>= 95%	pytest-cov + manual audit

8. Deployment

8.1 Environments

Environment	Purpose	Infrastructure
Local Dev	Feature development and debugging	docker-compose (Postgres on :5433, API on :8080, Frontend on :8888)
Staging	Pre-release integration testing and UAT	Render web service + managed Postgres; separate slug
Production	Live customer-facing system	Render web service (Docker) + Render Postgres 16; Vercel for frontend

8.2 CI/CD Pipeline

- Trigger: Push to main or merge of approved PR
- Step 1 — Lint: ESLint (frontend) + ruff/flake8 (backend)
- Step 2 — Test: pytest (backend) + vitest (frontend)
- Step 3 — Build: docker build using Dockerfile
- Step 4 — Deploy: Render detects Dockerfile and redeploys; scripts/start.sh runs Alembic upgrade head before uvicorn start
- Step 5 — Smoke test: smoke_test_http.py runs against staging; blocks production promotion on failure

8.3 Container Startup Sequence

11. 1. PostgreSQL health check passes (pg_isready -U postgres -d saas_platform, up to 20 retries)
12. 2. API container starts; scripts/start.sh executes python -m alembic -c alembic/alembic.ini upgrade head
13. 3. Alembic applies pending migrations against DATABASE_URL from environment
14. 4. uvicorn app.main:app starts; lifespan() calls Base.metadata.create_all() as a safety net
15. 5. /health endpoint returns {"status": "healthy"} — used by load balancer readiness probe

8.4 Production Configuration

Environment Variable	Description	Production Requirement
DATABASE_URL	PostgreSQL connection string (auto-normalised from postgres:// to postgresql+psycopg2://)	Required
SECRET_KEY	JWT signing secret; must be cryptographically random	Required; min 32 chars
ALGORITHM	JWT algorithm (default: HS256)	Optional

ACCESS_TOKEN_EXPIRE_MINUTES	JWT TTL in minutes (default: 14400 = 10 days)	Recommended: reduce in prod
PORT	Uvicorn listen port (default: 8000)	Set by Render automatically

9. Maintenance & Operations

9.1 Monitoring & Observability

- Request tracing: every response includes x-request-id header (generated by request_id_middleware); correlatable in AuditLog.details JSON
- Health endpoint: GET /health → {"status": "healthy"} for uptime monitoring and load-balancer probes
- Audit logs: queryable via GET /api/v1/audit-logs with tenant_id, user_id, and limit filters
- Application logs: uvicorn access logs captured by hosting platform (Render) log drain
- Database: Render managed Postgres provides automatic backups, connection pooling metrics, and slow query alerting

9.2 Incident Response

Severity	Definition	Response SLA	Escalation
P1 — Critical	Full platform outage; authentication broken; data loss risk	Acknowledge < 15 min; resolve < 2 h	Engineering Lead + CTO
P2 — High	Major feature broken (e.g. RBAC bypass); security vulnerability	Acknowledge < 1 h; resolve < 8 h	Engineering Lead
P3 — Medium	Non-critical feature degraded; performance regression	Acknowledge < 4 h; resolve next sprint	Developer on-call
P4 — Low	Cosmetic bug; documentation error	Backlog triage	Standard sprint process

9.3 Dependency Management

- Backend: requirements.txt pinned to major versions; monthly review of security advisories (pip-audit)
- Frontend: package-lock.json committed; npm audit run in CI; Dependabot or Renovate recommended
- Base image: python:3.11-slim reviewed quarterly for CVE patches

9.4 Database Operations

- Migrations: all schema changes via Alembic — never manual DDL in production
- Rollback: alembic downgrade -1 available but requires code rollback first
- Backups: Render automated daily backups; point-in-time recovery for >= Pro tier
- Data seeding: seed.py for default modules and permissions; create_sample_data.py for development fixtures

10. Risk Register

ID	Risk Description	Likelihood	Impact	Mitigation
R-001	Rate limiter state lost on process restart (in-memory store not distributed)	High	Medium	Replace SlidingWindowRateLimiter with Redis-backed solution for production multi-instance deployments
R-002	CORS allows all origins (allow_origins=['*']) — CSRF risk in browser contexts	Medium	High	Restrict allow_origins to known frontend domains in production environment configuration
R-003	SECRET_KEY defaults to 'change-me-in-prod' in dev config	Low	Critical	CI checks for default secret; Render generateValue: true in render.yaml enforces unique key
R-004	Admin role hardcoded superuser check — role rename could bypass RBAC	Low	High	Document that 'Admin' is a reserved role name; add DB constraint or validation on role creation
R-005	No distributed tracing — x-request-id not propagated to downstream services	Medium	Low	Acceptable for current single-service architecture; revisit when microservices are introduced
R-006	bcrypt 72-byte truncation for very long passwords	Low	Medium	validate_bcrypt_password_length() already implemented; returns 400 on violation
R-007	Alembic migration failure on deployment leaves DB in partial state	Low	High	Test migrations on staging before production; use transactional DDL where Postgres supports it

11. Change Management

11.1 Change Control Process

16. All changes proposed via GitHub Pull Request against main branch
17. PR must include: summary of change, test evidence, and any migration notes
18. Minimum 1 peer review approval required before merge
19. Breaking API changes require a version bump and deprecation notice
20. Schema changes require an accompanying Alembic migration
21. Security fixes (P1/P2) may bypass sprint cadence and merge immediately post-review

11.2 Version Numbering

The project follows Semantic Versioning (SemVer): MAJOR.MINOR.PATCH.

- MAJOR: breaking API contract changes or incompatible data model changes
- MINOR: new features, endpoints, or non-breaking model additions
- PATCH: bug fixes, performance improvements, documentation updates

The current API version is /api/v1. Minor/patch releases within v1 do not change the URL prefix.

11.3 Document Revision History

Version	Date	Author	Description
1.0	March 2026	Engineering Lead	Initial release — full SDLC documentation for MultiTenantPlatform v1.0

12. Document Approval

The following parties have reviewed and approved this SDLC document:

Role	Name	Signature	Date
Engineering Lead			
Product Owner			
QA Lead			
DevOps / Infra Lead			

END OF DOCUMENT